

CLOUD COMPUTING

Laboratory Instruction Guide

Computer Science related undergraduate programs

Three Practical Areas — 14 Lab Sessions

Area 1: Virtualization and Virtual Machines (Labs 1 – 4)

Area 2: Containers and Microservices (Labs 5 – 9)

Area 3: Private Cloud with OpenStack (Labs 10 – 14)

Based on the official OpenStack Install Guide (2024.2 Dalmatian)

Table of Contents

Table of Contents	2
Introduction	5
Area 1: Virtualization and Virtual Machines	6
1.1 Install VirtualBox	6
1.2 Create the Host-Only Network	6
1.3 Download the Ubuntu 22.04 Server ISO	7
1.4 Create the Virtual Machine	7
1.5 Configure Both Network Adapters	7
1.6 Attach the ISO and Install Ubuntu	7
1.7 First Boot and Interface Identification	8
1.8 Verify Host-to-VM Connectivity	8
1.9 System Update and Essential Package Installation	8
2.1 Adjust Hardware Settings	11
2.2 Take a Snapshot	11
2.3 Restore a Snapshot	11
2.4 Add a Second Virtual Disk	11
2.5 Format, Mount, and Persist the Disk	11
3.1 Networking Mode Reference	13
3.2 Observe NAT Behaviour	13
3.3 Use the Host-Only Adapter for Host Access	13
3.4 Observe Bridged Mode	14
3.5 Finalise the Golden Image and Take a Snapshot	14
3.6 Clone and Build the Host-Only Network	14
4.1 Export and Import (Cold Migration)	16
4.2 Full Clone vs Linked Clone	16
4.3 Resource Monitoring from the VirtualBox Host	16
4.4 Resource Monitoring Inside the VM	16
Area 2: Containers and Microservices	18
5.1 Install Docker Engine	18
5.2 Working with Images	19
5.3 Running and Managing Containers	19
5.4 Container Lifecycle and Commit	19
6.1 Create the Application	20
6.2 Write the Dockerfile	20
6.3 Build and Run	20

6.4 Push to Docker Hub	21
7.1 Create docker-compose.yml	22
7.2 Start and Inspect.....	22
7.3 Service DNS and Database Connectivity	22
7.4 Scale and Observe Port Conflicts	23
7.5 Tear Down	23
8.1 Install kubectl	24
8.2 Install and Start Minikube	24
8.3 Create a Deployment	24
8.4 Expose as a Service	25
8.5 Scale and Rolling Update.....	25
9.1 Prepare the Storage Path in Minikube.....	26
9.2 Create PersistentVolume and PersistentVolumeClaim.....	26
9.3 ConfigMaps.....	26
9.4 Secrets	27
9.5 Deploy a Pod Using the PVC, ConfigMap, and Secret	27
Area 3: Private Cloud with OpenStack	29
Node Architecture.....	29
Preparing the Two OpenStack Nodes from the Golden Image	29
Step 1 — Create the Controller Node VM	29
Step 2 — Create the Compute Node VM	30
Step 3 — Verify Distinct Identities	30
10.1 Disable the Firewall — Both Nodes.....	31
10.2 Host Networking — Controller Node	31
10.3 Host Networking — Compute Node	32
10.4 Verify Inter-Node Connectivity.....	33
10.5 NTP — Controller Node	33
10.6 NTP — Compute Node	33
10.7 OpenStack Packages — Both Nodes.....	33
10.8 SQL Database (MariaDB) — Controller Only	33
10.9 Message Queue (RabbitMQ) — Controller Only	34
10.10 Memcached — Controller Only	34
10.11 Etcd — Controller Only	34
11.1 Create the Keystone Database	36
11.2 Install and Configure Keystone	36
11.3 Bootstrap Keystone.....	36

11.4 Configure Apache HTTP Server.....	36
11.5 Create Admin Credentials File	37
11.6 Create Service Project, Demo User, and Role	37
11.7 Verify Keystone.....	37
12.1 Create the Glance Database and Credentials	38
12.2 Install and Configure Glance	38
12.3 Upload a Cloud Image	39
12.4 Install and Configure Placement	39
13.1 Nova — Controller Node	41
13.2 Nova — Compute Node	42
13.3 Enable Kernel Networking Modules — Both Nodes	43
13.4 Neutron — Controller Node.....	44
13.5 Neutron — Compute Node.....	45
13.6 Verify Nova and Neutron.....	46
13.7 Install Horizon Dashboard — Controller Only.....	47
14.1 Create the Provider Network.....	48
14.2 Create a Flavor	48
14.3 Create Key Pair and Security Group Rules	48
14.4 Launch Instance 1 — Web Server.....	48
14.5 Launch Instance 2 — Database Server.....	49
14.6 Verify Console Access	49
14.7 Verify SSH Access from the Controller.....	49
14.8 Explore via Horizon.....	49
Appendix A: Password Reference.....	51
Appendix B: Service Port Reference	51
Appendix C: Common Troubleshooting.....	52

Introduction

This Laboratory Instruction Guide covers three interconnected areas of cloud computing practice for undergraduate students. Labs are numbered sequentially from 1 to 14, each building on skills from preceding sessions.

Area	Labs	Topic	Key Skills
Area 1	1 – 4	Virtualization and Virtual Machines	VirtualBox, dual-adapter setup, VM config, networking, snapshots, migration
Area 2	5 – 9	Containers and Microservices	Docker, Dockerfile, Compose, Kubernetes, persistent storage
Area 3	10 – 14	Private Cloud with OpenStack	Controller node, Compute node, Keystone, Glance, Nova, Neutron, Horizon

Hardware

Each workstation requires at least 8 GB RAM (16 GB recommended), 4 CPU cores with VT-x/AMD-V enabled in BIOS, 100 GB free disk, and a wired LAN connection. Area 3 requires two machines (physical or VMs) on the same network — one Controller and one Compute node.

OS & Prompts

All labs use Ubuntu 22.04 LTS. Commands shown with \$ may be run by any user. Commands shown with # require root or sudo. Interface names (enp0s3, enp0s8) are examples — always check your actual interface name with: `ip link show`

Standard Adapter Setup for Area 1 Labs

Every VM in this course uses TWO network adapters:

Adapter 1 (NAT): Provides internet access for package downloads. VirtualBox assigns 10.0.2.15.

The host CANNOT ping or SSH into the VM through this adapter — this is normal.

Adapter 2 (Host-Only): Provides a reachable IP (e.g. 192.168.56.x) for host-to-VM and VM-to-VM communication. Always use this IP when pinging or SSH-ing from the host.

Set this up in Settings > Network before starting any VM. Both adapters must be configured.

Area 1: Virtualization and Virtual Machines

Virtualization is the foundation of all cloud computing. A hypervisor abstracts physical hardware so multiple operating system instances — called virtual machines (VMs) — can run in isolated environments on a single physical host. Area 1 uses VirtualBox, a Type 2 hosted hypervisor.

Concept	Description
Type 2 Hypervisor	Runs on top of a host OS. Examples: VirtualBox, VMware Workstation. Hardware access is mediated through the host OS.
NAT Adapter	VM gets internet via the host. VirtualBox assigns 10.0.2.15. The host CANNOT reach the VM on this IP.
Host-Only Adapter	Creates a private network between the host and VMs. The host CAN reach the VM on this IP. No internet from this adapter alone.
Snapshot	A point-in-time capture of a VM's disk and optionally RAM, enabling fast rollback.
OVA	Open Virtualisation Appliance — a portable archive of a VM's disk and configuration.

Lab 1: Installing VirtualBox, Creating a VM, and Building the Golden Image

Duration: 3 hours

Objectives

- Install VirtualBox and the Extension Pack.
- Configure a Host-Only network adapter for host-to-VM connectivity.
- Create a Ubuntu 22.04 Server VM with NAT and Host-Only adapters.
- Complete the Ubuntu installation and verify connectivity from the host.
- Update the system and install all essential packages used across the course.
- Disable cloud-init network management and prepare the VM as a golden image for cloning.

1.1 Install VirtualBox

1. Download VirtualBox 7.x from <https://www.virtualbox.org/wiki/Downloads> for your host OS (Windows, macOS, or Linux).
2. Run the installer and accept all defaults. Also install the VirtualBox Extension Pack when prompted — this adds USB 3.0, RDP, and disk encryption support.
3. Launch VirtualBox and confirm the main window opens without errors.

1.2 Create the Host-Only Network

This step must be done once. It creates the private network that allows the host machine to communicate with VMs.

4. In VirtualBox go to File > Host Network Manager (Windows/Linux) or VirtualBox > Preferences > Network (macOS).

5. Click the + icon to create a new Host-Only adapter. It will be named vboxnet0 (Linux/macOS) or VirtualBox Host-Only Ethernet Adapter (Windows).
6. Set the adapter IP to 192.168.56.1, mask 255.255.255.0. Leave the DHCP Server tab enabled with these settings: Server Address 192.168.56.100, Lower bound 192.168.56.101, Upper bound 192.168.56.254.
7. Click OK to save.

1.3 Download the Ubuntu 22.04 Server ISO

8. Download the Ubuntu Server 22.04 LTS ISO from <https://ubuntu.com/download/server> (approximately 1.4 GB). Save it to a dedicated folder such as ~/ISOs/.

1.4 Create the Virtual Machine

9. In VirtualBox click New. Set the name to Ubuntu-Lab1, Type to Linux, Version to Ubuntu (64-bit). Click Next.
10. Set memory to 2048 MB (2 GB). Click Next.
11. Select Create a virtual hard disk now, choose VDI, select Dynamically allocated, set size to 20 GB. Click Create.

1.5 Configure Both Network Adapters

WARNING

Do this before starting the VM for the first time. The default single NAT adapter gives the VM internet access but is unreachable from the host. You need both adapters.

12. Select Ubuntu-Lab1 and open Settings > Network.
13. Adapter 1 tab: confirm it is set to NAT. This is the default and gives the VM internet access.
14. Adapter 2 tab: tick Enable Network Adapter. Set Attached to: Host-only Adapter. Select your vboxnet0 adapter from the Name dropdown. Click OK.

1.6 Attach the ISO and Install Ubuntu

15. Open Settings > Storage. Under Controller: IDE click the empty disc icon. Click the disc icon beside Optical Drive on the right and select the Ubuntu ISO.
16. Go to Settings > System > Boot Order. Ensure Optical is above Hard Disk. Click OK.
17. Start the VM. Follow the Ubuntu Server installer:
 - Accept default partition layout.
 - Set a username (e.g. student) and a memorable password.
 - On the SSH Setup screen: select Install OpenSSH server. This is required for SSH access in later labs.
 - Proceed through remaining screens and confirm the installation.
18. After installation the VM reboots. Remove the ISO: Settings > Storage, click the ISO entry, choose Remove Disk from Virtual Drive. Click OK.

1.7 First Boot and Interface Identification

19. Start the VM and log in at the console with the username and password you set.

20. Identify your network interfaces:

```
ip link show
```

You should see three interfaces: lo (loopback), and two named interfaces such as enp0s3 (NAT, Adapter 1) and enp0s8 (Host-Only, Adapter 2). The exact names depend on your hardware — record them now.

21. Check the IP addresses:

```
ip addr show
```

enp0s3 will show 10.0.2.15/24 — this is the NAT address. enp0s8 will show 192.168.56.x/24 (assigned by the Host-Only DHCP server) — this is the address you use for host-to-VM access.

1.8 Verify Host-to-VM Connectivity

22. Record the enp0s8 IP address shown in the previous step (e.g. 192.168.56.101).

23. From your host machine open a terminal and ping the Host-Only IP:

```
ping 192.168.56.101 # use your actual enp0s8 IP
```

24. Confirm you receive replies. If there are no replies, check that Adapter 2 is set to Host-Only in VirtualBox settings and that the DHCP server is enabled on vboxnet0.

25. SSH into the VM from the host:

```
ssh student@192.168.56.101 # replace with your enp0s8 IP
```

26. Inside the VM, confirm internet access via the NAT adapter:

```
ping -c 4 8.8.8.8
```

NOTE

Always use the enp0s8 (Host-Only) IP when connecting from the host. The enp0s3 IP (10.0.2.15) is behind VirtualBox's NAT engine and cannot be reached from outside the VM.

1.9 System Update and Essential Package Installation

Before using the VM for any lab work, update the package lists, apply all available upgrades, and install the essential tools used across all three areas of this course. Doing this now — while the golden image is being built — means every clone inherits an up-to-date, fully equipped base.

27. Update package lists and upgrade all installed packages:

```
sudo apt update && sudo apt upgrade -y
```

28. Install core networking and diagnostic tools:

```
sudo apt install -y net-tools iproute2 iputils-ping traceroute curl wget dnsutils netcat-openbsd
```

29. Install netplan and its rendering backend (required for static IP configuration in Labs 3 and Area 3):

```
sudo apt install -y netplan.io systemd-resolved network-manager
```

30. Install system monitoring and disk utilities used in Labs 2 and 4:

```
sudo apt install -y htop sysstat iotop lsof tree nano vim
```

31. Install virtualisation guest utilities so VirtualBox can interact with the VM (clipboard sharing, dynamic screen resize, drag-and-drop):

```
sudo apt install -y virtualbox-guest-utils virtualbox-guest-additions-iso
```

NOTE

If virtualbox-guest-utils is not found in the repository, install the guest additions from the VirtualBox menu instead: Devices > Insert Guest Additions CD image, then run: `sudo mount /dev/cdrom /mnt && sudo /mnt/VBoxLinuxAdditions.run`

32. Install cloud-init tools so you can manage and suppress cloud-init behaviour cleanly (needed for Lab 3 golden image preparation):

```
sudo apt install -y cloud-init cloud-initramfs-dyn-netconf
```

33. Ensure the cloud-init network configuration is disabled so it does not overwrite your netplan files on reboot. Create the disable file now:

```
sudo mkdir -p /etc/cloud/cloud.cfg.d
echo 'network: {config: disabled}' | sudo tee /etc/cloud/cloud.cfg.d/99-disable-
network-config.cfg
```

34. Install OpenSSH server if it was not selected during the Ubuntu installer (verify first):

```
sudo systemctl status ssh          # if inactive or not found, install it:
sudo apt install -y openssh-server
sudo systemctl enable ssh && sudo systemctl start ssh
```

35. Remove the cloud-init generated netplan file and replace it with your own permanent one. This is done now so every clone inherits the correct netplan setup:

```
sudo rm -f /etc/netplan/50-cloud-init.yaml
sudo nano /etc/netplan/01-lab-network.yaml
```

Add the following content:

```
network:
  version: 2
  ethernet:
    enp0s3:
      dhcp4: true
    enp0s8:
      dhcp4: true
sudo chmod 600 /etc/netplan/01-lab-network.yaml
sudo netplan apply
ip addr show enp0s8          # confirm enp0s8 still has 192.168.56.x
```

36. Verify all installed tools are working:

```
htop --version
netstat -tulnp 2>/dev/null | head # from net-tools
ip route show                     # from iproute2
```

37. Clean up downloaded package files to reduce the image size before snapshotting:

```
sudo apt autoremove -y && sudo apt clean
```

38. Record the final state of the system for your lab report:

```
uname -a          # kernel version
lsb_release -a    # Ubuntu version
df -h             # disk usage
free -h           # memory
dpkg -l | wc -l   # number of installed packages
```

**Lab Report
Q1**

Explain what a Type 2 hypervisor is and how it differs from a Type 1 hypervisor. Why does VirtualBox's NAT adapter assign 10.0.2.15 to every VM, and why can the host not ping that address? What purpose does the Host-Only adapter serve? Why is it important to disable cloud-init network configuration before cloning a VM?

Lab 2: VM Configuration — CPU, Memory, Storage, and Snapshots

Duration: 2 hours

Objectives

- Adjust vCPU and memory allocation and verify the change inside the VM.
- Take and restore snapshots to understand rollback behaviour.
- Add a second virtual disk, format it, mount it, and make the mount persistent across reboots.

2.1 Adjust Hardware Settings

39. Power off Ubuntu-Lab1 cleanly:

```
sudo shutdown -h now
```

40. Open Settings > System > Processor. Set vCPUs to 2 and tick Enable PAE/NX. Click OK.

41. Start the VM, log in, and confirm the change:

```
nproc          # should now report 2
free -h        # confirm 2 GB RAM is visible
```

2.2 Take a Snapshot

42. With the VM running, go to Machine > Take Snapshot. Name it Clean-Install. Add the description: Fresh Ubuntu install, no customisations. Click OK.

43. Install a package to change the VM state:

```
sudo apt update && sudo apt install -y htop
```

44. Confirm htop works — run htop and press q to quit. Take a second snapshot named After-htop-install.

2.3 Restore a Snapshot

45. Power off the VM. In VirtualBox open Machine > Snapshots. Right-click Clean-Install and choose Restore Snapshot. Confirm when prompted.

46. Start the VM. Try to run htop — it should be reported as not found, confirming the rollback worked.

NOTE

A snapshot rollback discards everything that happened after the snapshot was taken — installed packages, created files, and configuration changes are all removed.

2.4 Add a Second Virtual Disk

47. Power off the VM. Go to Settings > Storage. Click the + icon next to Controller: SATA. Choose Add Hard Disk > Create New Disk. Select VDI, Dynamically allocated, set size to 5 GB. Name it Lab2-Disk2.vdi and click Create.

48. Start the VM. Verify the new disk appears:

```
lsblk
```

You should see sda (the primary 20 GB disk) and sdb (the new 5 GB disk). If sdb does not appear, reboot the VM.

2.5 Format, Mount, and Persist the Disk

49. Format the disk with the ext4 filesystem:

```
sudo mkfs.ext4 /dev/sdb
```

50. Create a mount point and mount the disk:

```
sudo mkdir -p /mnt/data
sudo mount /dev/sdb /mnt/data
df -h | grep sdb          # confirm mount and available space
```

51. Write a test file to confirm the disk is writable:

```
echo 'Lab 2 disk test' | sudo tee /mnt/data/testfile.txt
sudo cat /mnt/data/testfile.txt
```

52. Make the mount persistent so it survives a reboot. First get the disk's UUID:

```
sudo blkid /dev/sdb
```

Note the UUID value (a string like 6a8e3c1f-...). Now edit `/etc/fstab`:

```
sudo nano /etc/fstab
```

Add this line at the bottom, replacing the UUID with your actual value:

```
UUID=your-uuid-here /mnt/data ext4 defaults 0 2
```

53. Test the fstab entry before rebooting:

```
sudo umount /mnt/data
sudo mount -a          # mounts everything in fstab
df -h | grep sdb       # confirm it remounted
```

54. Reboot and verify the mount persists:

```
sudo reboot
df -h | grep sdb       # sdb should still be mounted after reboot
```

**Lab Report
Q2**

Describe what happens to installed software and created files when a snapshot is restored. What is the difference between a snapshot and a full clone? What is `/etc/fstab` and why is it necessary for a disk mount to survive a reboot?

Lab 3: VM Networking — NAT, Bridged, and Host-Only in Depth

Duration: 2 hours

Objectives

- Understand and demonstrate the behaviour of NAT, Bridged, and Host-Only adapter modes.
- Understand which adapter to use for which purpose.
- Prepare a golden image by removing cloud-init network files and clearing the machine-id.
- Clone the golden image and confirm each clone receives a unique IP address.

3.1 Networking Mode Reference

Mode	VM gets internet?	Host can reach VM?	VMs reach each other?	Best used for
NAT	Yes (via host)	No — host cannot reach 10.0.2.x	No	Outbound internet from any VM
Bridged	Yes (own LAN IP)	Yes	Yes (if on same LAN)	Simulating a real networked server
Host-Only	No (unless combined with NAT)	Yes — host can reach 192.168.56.x	Yes	Lab networks, host-to-VM SSH
Internal	No	No	Yes	Fully isolated inter-VM networks

NOTE

In this course every VM uses NAT (Adapter 1) + Host-Only (Adapter 2). The NAT adapter provides internet. The Host-Only adapter provides host access. This combination gives the best of both worlds.

3.2 Observe NAT Behaviour

55. With Ubuntu-Lab1 running (both adapters configured), run from inside the VM:

```
ping -c 4 8.8.8.8           # internet works via NAT
curl -s https://ifconfig.me # shows host's public IP, not VM's
ip addr show enp0s3         # shows 10.0.2.15 — the NAT address
```

56. From the host, try to ping the NAT address:

```
ping 10.0.2.15             # no reply — expected behaviour
```

This confirms that the NAT adapter provides outbound internet access only. The host cannot initiate connections to the VM through NAT.

3.3 Use the Host-Only Adapter for Host Access

57. From inside the VM, note the enp0s8 IP:

```
ip addr show enp0s8         # shows 192.168.56.x
```

58. From the host, ping the enp0s8 address:

```
ping 192.168.56.101        # replace with your actual IP — should reply
```

59. SSH from the host:

```
ssh student@192.168.56.101
```

This is the correct way to access a VM from the host throughout this course.

3.4 Observe Bridged Mode

60. Power off Ubuntu-Lab1. Change Adapter 1 from NAT to Bridged Adapter. Select your host's active physical NIC from the Name dropdown. Start the VM.

```
ip addr show enp0s3 # now shows a LAN IP (e.g. 192.168.1.x) from your router
```

61. From a separate machine on the same LAN, ping the VM's bridged IP. Confirm it replies. This shows the VM is now a visible node on the physical network.
62. Revert to NAT before the next section: power off, change Adapter 1 back to NAT, start the VM.

3.5 Finalise the Golden Image and Take a Snapshot

The cloud-init network file was already removed and 01-lab-network.yaml was already created in Lab 1 section 1.9. The only remaining step before cloning is to clear the machine identity so every clone boots with a unique DHCP client ID.

63. Verify the cloud-init network disable file is in place and 50-cloud-init.yaml is gone:

```
cat /etc/cloud/cloud.cfg.d/99-disable-network-config.cfg # should print: network:
{config: disabled}
ls /etc/netplan/ # should show 01-lab-network.yaml only - no 50-cloud-
init.yaml
```

64. If 50-cloud-init.yaml is still present (e.g. the VM was not updated in Lab 1), remove it now:

```
sudo rm -f /etc/netplan/50-cloud-init.yaml
sudo netplan apply
```

65. Clear the machine-id. Leave the file empty — do not delete it and do not run systemd-machine-id-setup. Systemd will generate a fresh unique ID on the next boot of every clone:

```
sudo truncate -s 0 /etc/machine-id
sudo rm -f /var/lib/dbus/machine-id
cat /etc/machine-id # should be empty (zero bytes)
```

WARNING

Do NOT run systemd-machine-id-setup after truncating. Leave the file empty. If you run it now the ID gets fixed again and all clones will share it.

66. Shut down Ubuntu-Lab1 cleanly:

```
sudo shutdown -h now
```

67. Take the Golden-Image snapshot in VirtualBox before cloning:

Select Ubuntu-Lab1 in VirtualBox. Go to Machine > Take Snapshot. Name it Golden-Image. Add the description: Clean Ubuntu 22.04, all packages installed, cloud-init suppressed, machine-id cleared.

3.6 Clone and Build the Host-Only Network

68. Clone Ubuntu-Lab1 from the Golden-Image snapshot: right-click > Clone. Name it Ubuntu-Lab3-B. Select Full clone and tick Reinitialise MAC addresses of all network cards. Click Clone.
69. Verify both VMs have Adapter 1 = NAT and Adapter 2 = Host-Only (vboxnet0). Start both VMs.
70. On first boot, systemd generates a fresh machine-id on Ubuntu-Lab3-B, the Host-Only DHCP server sees a new client, and assigns a different IP. Check both:

```
# Inside Ubuntu-Lab1:
ip addr show enp0s8          # e.g. 192.168.56.101
# Inside Ubuntu-Lab3-B:
ip addr show enp0s8          # e.g. 192.168.56.102 - must be different
```

NOTE

If both VMs show the same enp0s8 IP, power off Ubuntu-Lab3-B and run: `sudo truncate -s 0 /etc/machine-id && sudo rm -f /var/lib/dbus/machine-id` — then reboot. Do not run `systemd-machine-id-setup`.

71. From Ubuntu-Lab1, ping Ubuntu-Lab3-B using its enp0s8 IP:

```
ping -c 4 192.168.56.102    # replace with actual IP of Ubuntu-Lab3-B
```

72. Install openssh-server on Ubuntu-Lab3-B if it was not selected during setup:

```
sudo apt install -y openssh-server
```

73. From Ubuntu-Lab1, SSH into Ubuntu-Lab3-B:

```
ssh student@192.168.56.102
```

**Lab Report
Q3**

Draw a network diagram showing both VMs with their two adapters, labelled with IP addresses and adapter modes. Explain why cloned VMs get the same IP when machine-id and cloud-init network files are not cleared. For each of the four adapter modes (NAT, Bridged, Host-Only, Internal), describe a real-world scenario where it is the correct choice.

Lab 4: VM Portability, Cloning, and Resource Monitoring

Duration: 2 hours

Objectives

- Export a VM as OVA and re-import it to demonstrate cold migration.
- Compare full clone and linked clone in size and independence.
- Monitor resource usage from inside the VM and from the VirtualBox host panel.

4.1 Export and Import (Cold Migration)

74. Power off Ubuntu-Lab1. Go to File > Export Appliance. Select Ubuntu-Lab1. Set format to OVA. Choose an output path and click Export. Note the file size and how long it takes.
75. Import it back: File > Import Appliance. Browse to the .ova file. Change the VM name to Ubuntu-Imported. Accept defaults and click Import.
76. Start Ubuntu-Imported. Log in and confirm the testfile from Lab 2 is still present:

```
cat /mnt/data/testfile.txt
```

The full disk and filesystem state was preserved through export and import.

4.2 Full Clone vs Linked Clone

77. Right-click Ubuntu-Lab1 > Clone. Select Full clone. Name it Ubuntu-Full-Clone. Tick Reinitialise MAC addresses. Click Clone.
78. Repeat, this time selecting Linked clone. Name it Ubuntu-Linked-Clone.
79. Compare disk sizes on the host (run from a host terminal):

```
du -sh ~/VirtualBox\ VMs/Ubuntu-Full-Clone/
du -sh ~/VirtualBox\ VMs/Ubuntu-Linked-Clone/
```

The linked clone will be several gigabytes smaller because it shares the base disk with the parent VM and only stores changes.

4.3 Resource Monitoring from the VirtualBox Host

80. Start Ubuntu-Lab1. Open Machine > Session Information (or press Host+N) to see real-time CPU, disk I/O, and network graphs.
81. Install the stress tool and generate CPU load:

```
sudo apt install -y stress
stress --cpu 2 --timeout 30
```

82. Observe the CPU graph spike in the Session Information panel during the 30 seconds.

4.4 Resource Monitoring Inside the VM

83. Install monitoring tools if not present:

```
sudo apt install -y htop sysstat
htop                                # interactive process view - q to quit
vmstat 1 10                         # CPU, memory, I/O every second, 10 samples
iostat -x 1 5                       # disk utilisation stats
```

**Lab Report
Q4**

What is cold migration and how does it differ from live migration? Compare the full clone and linked clone — when is each appropriate? Run three 2 GB VMs simultaneously on your 8 GB host and record the vmstat output. What happens to the idle CPU percentage?

Area 2: Containers and Microservices

Containers share the host OS kernel and use Linux namespaces and cgroups to isolate processes. They are lighter than VMs, start in seconds, and package an application with all its dependencies into a portable image. Microservices architecture decomposes applications into small, independently deployable services, each running in its own container.

Feature	Virtual Machine	Container
Isolation	Full OS (separate kernel)	Process-level (shared kernel)
Startup time	1 – 3 minutes	Milliseconds to seconds
Image size	Gigabytes	Megabytes
Overhead	High (full OS per VM)	Low (shared kernel)
Best for	Full OS environments, legacy apps	Microservices, CI/CD, cloud-native apps

Lab 5: Introduction to Docker — Images and Containers

Duration: 2 hours

Objectives

- Install Docker Engine on Ubuntu.
- Pull images, run containers, and manage their lifecycle.
- Commit a modified container as a new image.

5.1 Install Docker Engine

1. Remove any old Docker packages to avoid conflicts:

```
sudo apt remove -y docker docker-engine docker.io containerd runc 2>/dev/null; true
```

2. Install prerequisite packages:

```
sudo apt update
sudo apt install -y ca-certificates curl gnupg
```

3. Add Docker's official GPG key and repository:

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null
```

4. Install Docker Engine:

```
sudo apt update && sudo apt install -y docker-ce docker-ce-cli containerd.io docker-
compose-plugin
```

5. Add your user to the docker group so you can run docker without sudo. You must log out and back in (or start a new shell with newgrp) for this to take effect:

```
sudo usermod -aG docker $USER
newgrp docker # applies the group change to the current shell
```

6. Verify the installation:

```
docker --version
docker run hello-world
```

5.2 Working with Images

```
docker search nginx          # search Docker Hub for nginx images
docker pull nginx:latest     # download the image
docker images                # list downloaded images – note size and ID
docker inspect nginx         # full image metadata in JSON
docker history nginx         # show the image layer history
```

5.3 Running and Managing Containers

7. Start nginx in the background and map host port 8080 to container port 80:

```
docker run -d -p 8080:80 --name webserver nginx
docker ps                    # confirm running status
curl http://localhost:8080   # should return the nginx welcome HTML
docker logs webserver        # view nginx access logs
docker exec -it webserver bash # open a shell inside the running container
exit                        # exit the container shell
docker stop webserver && docker rm webserver
```

5.4 Container Lifecycle and Commit

```
docker run -it --name test-box ubuntu:22.04 bash
# Inside the container:
apt update && apt install -y curl
exit
docker ps -a                # test-box shows as Exited
docker start -ai test-box    # restart – curl should still be installed
exit
docker commit test-box my-ubuntu:v1 # save state as a new image
docker images                # my-ubuntu:v1 should appear
docker run -it my-ubuntu:v1 bash  # curl is present in the new image
```

Lab Report Q5

Explain the difference between a Docker image and a container. What is the layer system in Docker images, and why does it make image distribution efficient? What happens to changes made inside a container when it is removed with `docker rm`?

Lab 6: Writing Dockerfiles — Containerising an Application

Duration: 2 hours

Objectives

- Write a Flask application and a Dockerfile to containerise it.
- Build and tag a Docker image, and understand the build cache.
- Run multiple instances and observe process isolation.
- Push the image to Docker Hub.

6.1 Create the Application

```
mkdir ~/flask-app && cd ~/flask-app
```

8. Create app.py:

```
from flask import Flask
import socket
app = Flask(__name__)
@app.route('/')
def home():
    return f'<h1>Hello from Container!</h1><p>Hostname: {socket.gethostname()}</p>'
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

9. Create requirements.txt:

```
flask==3.0.0
```

10. Create .dockerignore to prevent unnecessary files being copied into the image:

```
__pycache__
*.pyc
.git
```

6.2 Write the Dockerfile

11. Create a file named Dockerfile (no extension) in ~/flask-app/:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

NOTE

requirements.txt is copied and installed BEFORE the rest of the application code. This allows Docker to cache the pip install layer. If only app.py changes on a rebuild, Docker reuses the cached layer — saving minutes of build time.

6.3 Build and Run

```
docker build -t flask-app:v1 .
docker run -d -p 5000:5000 --name flask1 flask-app:v1
curl http://localhost:5000 # shows hostname of flask1 container
```

12. Run two more instances on different host ports:

```
docker run -d -p 5001:5000 --name flask2 flask-app:v1
docker run -d -p 5002:5000 --name flask3 flask-app:v1
```

```
curl http://localhost:5001          # different hostname – different container
curl http://localhost:5002          # different hostname again
```

13. Build a v2 image with a minor change (you will need this in Lab 8):

```
sed -i "s/Hello from Container!/Hello from Container v2!/" app.py
docker build -t flask-app:v2 .
```

6.4 Push to Docker Hub

14. Create a free account at <https://hub.docker.com> if you do not have one.

```
docker login
docker tag flask-app:v1 <your-username>/flask-app:v1
docker tag flask-app:v2 <your-username>/flask-app:v2
docker push <your-username>/flask-app:v1
docker push <your-username>/flask-app:v2
```

NOTE

Both v1 and v2 must be pushed to Docker Hub. Lab 8 uses the v2 tag for a rolling update. If the image is not publicly available on Docker Hub, Kubernetes will fail to pull it with an ImagePullBackOff error.

**Lab Report
Q6**

Explain the purpose of each instruction in your Dockerfile. Why is requirements.txt copied before the application code? What does the .dockerignore file do? How does the build cache work, and what breaks it?

Lab 7: Docker Compose — Multi-Container Applications

Duration: 2 hours

Objectives

- Define a multi-service application in a docker-compose.yml file.
- Understand service DNS, the depends_on directive, and named volumes.
- Scale services and observe port conflicts.

7.1 Create docker-compose.yml

```
mkdir ~/todo-app && cd ~/todo-app
```

15. Create docker-compose.yml:

```
version: '3.8'
services:
  web:
    image: flask-app:v1
    ports:
      - "5000:5000"
    environment:
      - APP_ENV=production
    depends_on:
      - db
    restart: unless-stopped
  db:
    image: postgres:15
    environment:
      - POSTGRES_USER=appuser
      - POSTGRES_PASSWORD=secret
      - POSTGRES_DB=todos
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

NOTE

The web service uses flask-app:v1 which does not connect to the database — the db service demonstrates the volume and networking concepts. The DATABASE_URL variable from the previous version of this lab has been removed to avoid confusion about why the connection does not work.

7.2 Start and Inspect

```
docker compose up -d
docker compose ps           # both services should show Running
docker compose logs web     # view web service stdout
docker network ls           # note the auto-created network (todo-app_default)
docker volume inspect todo-app_pgdata # confirm volume exists on host
```

7.3 Service DNS and Database Connectivity

Docker Compose creates a private bridge network and registers each service name as a DNS hostname within it. The web service can reach the db service at hostname 'db' without knowing its IP address.

```
docker compose exec web ping -c 3 db      # web container can reach db by name
docker compose exec db psql -U appuser -d todos
# Inside psql:
\l          # list databases
\q          # quit
```

7.4 Scale and Observe Port Conflicts

```
docker compose up -d --scale web=3
```

This will fail with a port conflict because three containers cannot all bind to host port 5000. This is expected — it demonstrates why fixed host port mappings prevent scaling.

```
docker compose up -d --scale web=1      # scale back down to 1
```

7.5 Tear Down

```
docker compose stop      # stop containers, keep them
docker compose down      # remove containers, keep volumes
docker compose down -v   # remove containers AND volumes
```

Lab Report Q7

Explain how Docker Compose service names work as DNS hostnames within the application. What does `depends_on` guarantee and what does it NOT guarantee? Why does scaling the web service fail when a fixed host port is mapped? What is the correct approach for running multiple replicas behind a load balancer?

Lab 8: Introduction to Kubernetes — Pods, Deployments, and Services

Duration: 2.5 hours

Objectives

- Install kubectl and Minikube to create a local Kubernetes cluster.
- Deploy the Flask application using a Deployment manifest.
- Expose the Deployment as a NodePort Service.
- Scale replicas and perform a rolling update using the v2 image.

8.1 Install kubectl

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
kubectl version --client
```

8.2 Install and Start Minikube

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
```

16. Start Minikube with the docker driver and adequate memory:

```
minikube start --driver=docker --memory=3072 --cpus=2
```

NOTE

If minikube start fails with permission denied, ensure your user is in the docker group: `sudo usermod -aG docker $USER`, then log out and back in.

```
kubectl cluster-info
kubectl get nodes # should show minikube node in Ready state
```

8.3 Create a Deployment

17. Create deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: flask
  template:
    metadata:
      labels:
        app: flask
    spec:
      containers:
        - name: flask
          image: <your-username>/flask-app:v1
          ports:
            - containerPort: 5000
kubectl apply -f deployment.yaml
```

```
kubectl get pods -w          # watch pods start — Ctrl+C to stop watching
```

NOTE

Replace <your-username> with your actual Docker Hub username. If the image is private or the name is wrong, pods will show ImagePullBackOff status. Check with: `kubectl describe pod <pod-name>`

8.4 Expose as a Service

18. Create service.yaml:

```
apiVersion: v1
kind: Service
metadata:
  name: flask-service
spec:
  selector:
    app: flask
  ports:
    - port: 80
      targetPort: 5000
  type: NodePort
kubectl apply -f service.yaml
minikube service flask-service --url      # get the access URL
```

19. Open the URL in a browser. Refresh multiple times — the hostname in the response rotates between the two pod names, showing Kubernetes is load-balancing across replicas.

8.5 Scale and Rolling Update

```
kubectl scale deployment flask-deployment --replicas=5
kubectl get pods          # 5 pods should appear
```

20. Perform a rolling update to v2 (which you built and pushed in Lab 6):

```
kubectl set image deployment/flask-deployment flask=<your-username>/flask-app:v2
kubectl rollout status deployment/flask-deployment
```

21. Refresh the browser — pods are replaced one at a time with zero downtime. The response text changes to 'Hello from Container v2!' as each pod is updated.

22. Roll back to v1:

```
kubectl rollout undo deployment/flask-deployment
kubectl rollout status deployment/flask-deployment
```

Lab Report Q8

Describe the relationship between a Deployment, a ReplicaSet, and a Pod. What does a rolling update do step by step, and why is it important for production services? What happens automatically when a pod crashes inside a Deployment?

Lab 9: Kubernetes — Persistent Storage, ConfigMaps, and Secrets

Duration: 2 hours

Objectives

- Create a PersistentVolume and PersistentVolumeClaim and bind them.
- Inject configuration data using a ConfigMap.
- Inject sensitive data using a Secret and understand the security implications.
- Mount the PVC into a pod and verify data persists across pod restarts.

9.1 Prepare the Storage Path in Minikube

NOTE

A hostPath PersistentVolume uses a path on the Minikube node's filesystem (inside the Minikube VM), not the host machine's filesystem. The path must exist inside Minikube before the PV is created.

```
minikube ssh -- sudo mkdir -p /data/lab-pv
```

9.2 Create PersistentVolume and PersistentVolumeClaim

23. Create pv.yaml:

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: lab-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  hostPath:
    path: /data/lab-pv
```

24. Create pvc.yaml:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: lab-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 500Mi
kubectl apply -f pv.yaml && kubectl apply -f pvc.yaml
kubectl get pvc                # status should show Bound
```

9.3 ConfigMaps

```
kubectl create configmap app-config --from-literal=APP_ENV=production --from-
literal=LOG_LEVEL=info
kubectl describe configmap app-config
```

9.4 Secrets

```
kubectl create secret generic db-creds --from-literal=username=appuser --from-literal=password=s3cr3t
kubectl get secret db-creds -o yaml
```

25. The password value is base64-encoded, not encrypted. Decode it to confirm:

```
echo 'czNjcjN0' | base64 -d      # prints: s3cr3t
```

WARNING

Kubernetes Secrets stored in etcd are NOT encrypted by default. Anyone with kubectl get secret access can retrieve the plaintext. In production, enable etcd encryption at rest and use an external secret manager such as HashiCorp Vault or AWS Secrets Manager.

9.5 Deploy a Pod Using the PVC, ConfigMap, and Secret

26. Create storage-pod.yaml:

```
apiVersion: v1
kind: Pod
metadata:
  name: storage-test-pod
spec:
  containers:
  - name: ubuntu
    image: ubuntu:22.04
    command: ["sleep", "3600"]
    envFrom:
    - configMapRef:
        name: app-config
    env:
    - name: DB_PASS
      valueFrom:
        secretKeyRef:
          name: db-creds
          key: password
    volumeMounts:
    - mountPath: /mnt/persistent
      name: lab-storage
  volumes:
  - name: lab-storage
    persistentVolumeClaim:
      claimName: lab-pvc
kubectl apply -f storage-pod.yaml
kubectl get pod storage-test-pod      # wait for Running
```

27. Verify the pod has access to the PVC, ConfigMap, and Secret:

```
kubectl exec -it storage-test-pod -- bash
env | grep APP_ENV          # from ConfigMap
env | grep DB_PASS          # from Secret
echo 'Persistent data' > /mnt/persistent/test.txt
exit
```

28. Delete and recreate the pod — the data in /mnt/persistent should survive:

```
kubectl delete pod storage-test-pod
kubectl apply -f storage-pod.yaml
kubectl exec -it storage-test-pod -- cat /mnt/persistent/test.txt
```

Lab Report
Q9

What is the difference between a PersistentVolume and a PersistentVolumeClaim? Why is this abstraction useful for developers? Explain why Kubernetes Secrets are not genuinely secret by default and what steps should be taken in production.

Area 3: Private Cloud with OpenStack

OpenStack is a free, open-source cloud computing platform for building private and public cloud infrastructure. These labs follow the official OpenStack Install Guide (2024.2 Dalmatian) on Ubuntu 22.04 LTS, simplified to a two-node deployment: one Controller node and one Compute node. Block Storage and Object Storage nodes are omitted.

Node Architecture

Node	Hostname	Management IP	Provider IF	Services
Controller	controller	10.0.0.11	No IP (enp0s8)	Keystone, Glance, Placement, Nova API, Neutron server, Horizon, MariaDB, RabbitMQ, Memcached, Etc
Compute	compute1	10.0.0.31	No IP (enp0s8)	Nova Compute, Neutron Linux Bridge agent

Network Setup for Area 3

Each node needs TWO network interfaces:

Interface 1 (Management — enp0s3): Static IP on 10.0.0.0/24. All OpenStack service traffic flows here.

Interface 2 (Provider — enp0s8): No IP address. Neutron Linux Bridge uses this for VM external traffic.

In VirtualBox: Adapter 1 = Host-Only (vboxnet0 or a dedicated network), Adapter 2 = Bridged or a second Host-Only.

Both nodes must ping each other by hostname and IP before any OpenStack service is installed.

Disable ufw on both nodes before starting: `sudo ufw disable`

All commands in Area 3 are run as root (shown with #) unless marked otherwise.

Preparing the Two OpenStack Nodes from the Golden Image

Both OpenStack nodes are created by full-cloning the Golden-Image snapshot taken at the end of Lab 3. This gives you two identical, clean Ubuntu 22.04 machines in minutes — no fresh OS installation required. The clone process also reinitialises the MAC address and triggers a new machine-id on first boot, so each node receives a unique IP automatically.

Step 1 — Create the Controller Node VM

1. In VirtualBox, right-click Ubuntu-Lab1 and choose Clone. Set the name to openstack-controller. Select Full clone and tick Reinitialise MAC addresses of all network cards. Under Snapshots, select Golden-Image as the source. Click Clone.
2. Before starting the VM, open Settings and configure the following:

- System > Processor: set vCPUs to 2.
 - System > Motherboard: set RAM to at least 4096 MB (6144 MB recommended — the controller runs many services simultaneously).
 - Network > Adapter 1: change from NAT to Host-Only Adapter (vboxnet0). This becomes the management interface.
 - Network > Adapter 2: set to Bridged Adapter (select your physical NIC) or a second Host-Only network. This becomes the provider interface — Neutron will use it without assigning an IP.
3. Start openstack-controller. On first boot systemd generates a fresh machine-id and the Host-Only DHCP server assigns a temporary IP (192.168.56.x). Log in and set the hostname immediately so the shell prompt identifies this node correctly:

```
sudo hostnamectl set-hostname controller
exec bash                # reload shell to show new hostname in prompt
hostname                 # should print: controller
```

Step 2 — Create the Compute Node VM

4. Repeat the clone process. Right-click Ubuntu-Lab1, choose Clone. Set the name to openstack-compute1. Full clone, Reinitialise MAC addresses, Golden-Image snapshot source. Click Clone.
5. Open Settings for openstack-compute1:
 - System > Processor: set vCPUs to 2.
 - System > Motherboard: set RAM to at least 2048 MB (4096 MB recommended — nova-compute needs headroom to run instances).
 - Network > Adapter 1: Host-Only Adapter (vboxnet0).
 - Network > Adapter 2: Bridged Adapter or second Host-Only — same as the controller.
6. Start openstack-compute1. Log in and set the hostname:

```
sudo hostnamectl set-hostname compute1
exec bash
hostname                # should print: compute1
```

Step 3 — Verify Distinct Identities

Before touching any OpenStack configuration, confirm the two nodes are genuinely distinct. Open a terminal on each and run:

```
hostname                # controller on one, compute1 on the other
cat /etc/machine-id     # must be DIFFERENT on each node
ip addr show enp0s3     # different 192.168.56.x IPs from DHCP
```

WARNING

If both nodes show the same machine-id or the same IP on enp0s3, do NOT continue. On the affected node run: `sudo truncate -s 0 /etc/machine-id && sudo rm -f /var/lib/dbus/machine-id` — then reboot. Never run `systemd-machine-id-setup` at this point.

NOTE

The temporary 192.168.56.x DHCP addresses are only used until Lab 10 step 10.2 assigns the permanent static management IPs (10.0.0.11 for controller, 10.0.0.31 for

compute1). Once netplan apply runs with the static config, your SSH session over the old DHCP address will drop — reconnect using the new static IP.

Lab 10: Environment Preparation — Both Nodes

Duration: 2.5 hours

Objectives

- Configure static IPs and hostnames on both nodes.
- Confirm inter-node connectivity by hostname.
- Install and configure NTP (Chrony) on both nodes.
- Enable the OpenStack 2024.2 repository on both nodes.
- Install MariaDB, RabbitMQ, Memcached, and Etcd on the Controller.

10.1 Disable the Firewall — Both Nodes

Perform this on both the Controller and Compute nodes.

```
# On BOTH nodes:
sudo ufw disable
sudo ufw status           # should report: Status: inactive
```

10.2 Host Networking — Controller Node

Perform steps 10.2 and 10.3 on the Controller node only.

7. Identify your interface names:

```
ip link show
```

The management interface is typically enp0s3 and the provider interface enp0s8. Use your actual names in all configuration below.

8. Edit the netplan configuration file. On Ubuntu 22.04 it is usually /etc/netplan/00-installer-config.yaml:

```
sudo nano /etc/netplan/00-installer-config.yaml
```

Replace the entire file content with:

```
network:
  version: 2
  ethernet:
    enp0s3:
      dhcp4: false
      addresses:
        - 10.0.0.11/24
      routes:
        - to: default
          via: 10.0.0.1
      nameservers:
        addresses: [8.8.8.8, 8.8.4.4]
    enp0s8:
      dhcp4: false
```

Replace enp0s3 and enp0s8 with your actual interface names. The provider interface enp0s8 must have no IP address assigned.

```
sudo netplan apply
```

WARNING

Your SSH session over the old 192.168.56.x address will disconnect the moment netplan apply runs. This is expected — the management interface now has a static IP. Reconnect using: `ssh student@10.0.0.11`

```
ip addr show enp0s3          # confirm 10.0.0.11/24 — run after reconnecting
```

NOTE

The hostname was already set to 'controller' during the clone preparation steps. Verify it is still correct before continuing.

```
hostname                    # must print: controller
cat /etc/machine-id         # must be non-empty and unique
```

9. Edit `/etc/hosts`. First check for and remove any line that maps the hostname to 127.0.1.1:

```
sudo nano /etc/hosts
```

The file should contain:

```
127.0.0.1    localhost
# 127.0.1.1  controller  <-- comment out or delete this line if present
10.0.0.11    controller
10.0.0.31    compute1
```

10. Apply the hostname change immediately:

```
exec bash
```

10.3 Host Networking — Compute Node

Perform the following on the Compute node.

11. Edit the netplan config file with:

```
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: false
      addresses:
        - 10.0.0.31/24
      routes:
        - to: default
          via: 10.0.0.1
      nameservers:
        addresses: [8.8.8.8, 8.8.4.4]
    enp0s8:
      dhcp4: false
sudo netplan apply
```

WARNING

Your SSH session over the old 192.168.56.x address will disconnect when netplan apply runs. Reconnect using: `ssh student@10.0.0.31`

```
ip addr show enp0s3          # confirm 10.0.0.31/24 — run after reconnecting
```

NOTE

The hostname was already set to 'compute1' during the clone preparation steps. Verify before continuing.

```
hostname                    # must print: compute1
cat /etc/machine-id         # must be non-empty and different from controller
```

12. Edit `/etc/hosts` — add the same two entries as the controller, and remove any 127.0.1.1 mapping of compute1:

```
sudo nano /etc/hosts
```

The file should contain:

```
127.0.0.1    localhost
# 127.0.1.1  compute1  <-- comment out or delete if present
10.0.0.11    controller
10.0.0.31    compute1
exec bash
```

10.4 Verify Inter-Node Connectivity

```
# From controller:
ping -c 4 compute1      # must reply before continuing
ping -c 4 10.0.0.31
# From compute1:
ping -c 4 controller    # must reply before continuing
ping -c 4 10.0.0.11
```

WARNING

Do not proceed to any OpenStack service installation until both ping tests succeed in both directions, by both IP and hostname.

10.5 NTP — Controller Node

```
# On controller:
sudo apt install -y chrony
```

13. Edit `/etc/chrony/chrony.conf`. Find the existing pool or server lines and replace them with:

```
server 0.pool.ntp.org iburst
server 1.pool.ntp.org iburst
allow 10.0.0.0/24
sudo service chrony restart
chronyc sources -v          # * indicates the active time source
```

10.6 NTP — Compute Node

```
# On compute1:
sudo apt install -y chrony
```

14. Edit `/etc/chrony/chrony.conf`. Comment out or remove all existing pool/server lines and add:

```
server controller iburst
sudo service chrony restart
chronyc sources -v          # should show controller as the source
```

10.7 OpenStack Packages — Both Nodes

```
# On BOTH nodes:
sudo add-apt-repository cloud-archive:dalmatian
```

Press Enter to confirm when prompted.

```
sudo apt update && sudo apt dist-upgrade -y
sudo apt install -y python3-openstackclient
openstack --version          # confirm client is installed
```

10.8 SQL Database (MariaDB) — Controller Only

```
# On controller:
sudo apt install -y mariadb-server python3-pymysql
```

15. Create the OpenStack-specific MariaDB configuration file:

```
| sudo nano /etc/mysql/mariadb.conf.d/99-openstack.cnf
```

Add the following content:

```
| [mysqld]
bind-address = 10.0.0.11
default-storage-engine = innodb
innodb_file_per_table = on
max_connections = 4096
collation-server = utf8_general_ci
character-set-server = utf8
sudo service mysql restart
```

16. Verify MariaDB is listening on the management IP:

```
| ss -tlnp | grep 3306          # should show 10.0.0.11:3306
```

17. Secure the installation. On Ubuntu 22.04, MariaDB uses `unix_socket` authentication for the root user. Run the secure installation script and when asked to change the root password, press Enter to leave it using socket authentication:

```
| sudo mysql_secure_installation
```

Answer the prompts as follows: Enter for root password, No to switch to password auth, Yes to remove anonymous users, Yes to disallow remote root login, Yes to remove test database, Yes to reload privileges.

18. Verify MariaDB access:

```
| sudo mysql                  # should open MariaDB prompt without password
SHOW DATABASES;
EXIT;
```

10.9 Message Queue (RabbitMQ) — Controller Only

```
| # On controller:
sudo apt install -y rabbitmq-server
sudo systemctl enable rabbitmq-server
sudo rabbitmqctl add_user openstack RABBIT_PASS
sudo rabbitmqctl set_permissions openstack ".*" ".*" ".*"
sudo rabbitmqctl list_users    # confirm openstack user appears
```

NOTE

Replace `RABBIT_PASS` with a strong password and record it. This same password must appear in the `transport_url` setting in `nova.conf` and `neutron.conf` on all nodes.

10.10 Memcached — Controller Only

```
| # On controller:
sudo apt install -y memcached python3-memcache
```

19. Edit `/etc/memcached.conf`. Find the line that starts with `-l` and change it to:

```
| -l 10.0.0.11
```

This allows other nodes to reach Memcached over the management network.

```
| sudo service memcached restart
ss -tlnp | grep 11211          # confirm listening on 10.0.0.11
```

10.11 Etcd — Controller Only

```
| # On controller:
sudo apt install -y etcd
```

20. Edit /etc/default/etcd. Replace all existing content with:

```
ETCD_NAME="controller"
ETCD_DATA_DIR="/var/lib/etcd"
ETCD_INITIAL_CLUSTER_STATE="new"
ETCD_INITIAL_CLUSTER_TOKEN="etcd-cluster-01"
ETCD_INITIAL_CLUSTER="controller=http://10.0.0.11:2380"
ETCD_INITIAL_ADVERTISE_PEER_URLS="http://10.0.0.11:2380"
ETCD_ADVERTISE_CLIENT_URLS="http://10.0.0.11:2379"
ETCD_LISTEN_PEER_URLS="http://0.0.0.0:2380"
ETCD_LISTEN_CLIENT_URLS="http://10.0.0.11:2379"
sudo systemctl enable etcd
sudo systemctl restart etcd
sudo systemctl status etcd # confirm active (running)
```

**Lab Report
Q10**

What is the role of each supporting service (MariaDB, RabbitMQ, Memcached, Etcd) in an OpenStack deployment? Why must NTP be configured before installing OpenStack services? What would happen if the controller and compute nodes had clocks that differed by more than a few seconds?

Lab 11: Keystone — Identity Service

Duration: 2 hours

Objectives

- Create the Keystone database and configure the Identity service.
- Bootstrap Keystone and create the admin credentials file.
- Create service project, demo user, and custom role.
- Verify Keystone with the CLI.

11.1 Create the Keystone Database

```
# On controller, as root:
mysql
CREATE DATABASE keystone;
GRANT ALL PRIVILEGES ON keystone.* TO 'keystone'@'localhost' IDENTIFIED BY
'KEYSTONE_DBPASS';
GRANT ALL PRIVILEGES ON keystone.* TO 'keystone'@'%' IDENTIFIED BY 'KEYSTONE_DBPASS';
FLUSH PRIVILEGES;
EXIT;
```

11.2 Install and Configure Keystone

```
sudo apt install -y keystone
```

21. Edit `/etc/keystone/keystone.conf`. Find the `[database]` section and set:

```
[database]
connection = mysql+pymysql://keystone:KEYSTONE_DBPASS@controller/keystone
```

22. Find the `[token]` section and set:

```
[token]
provider = fernet
```

23. Populate the database and initialise fernet keys:

```
sudo su -s /bin/sh -c "keystone-manage db_sync" keystone
sudo keystone-manage fernet_setup --keystone-user keystone --keystone-group keystone
sudo keystone-manage credential_setup --keystone-user keystone --keystone-group
keystone
```

11.3 Bootstrap Keystone

```
sudo keystone-manage bootstrap --bootstrap-password ADMIN_PASS \
--bootstrap-admin-url http://controller:5000/v3/ \
--bootstrap-internal-url http://controller:5000/v3/ \
--bootstrap-public-url http://controller:5000/v3/ \
--bootstrap-region-id RegionOne
```

11.4 Configure Apache HTTP Server

24. Edit `/etc/apache2/apache2.conf` and add this line at the end:

```
ServerName controller
sudo service apache2 restart
```

25. Verify Keystone is responding via Apache:

```
curl http://controller:5000/v3/ # should return JSON with version info
```

11.5 Create Admin Credentials File

26. Create `~/admin-openrc` with the following content:

```
export OS_PROJECT_DOMAIN_NAME=Default
export OS_USER_DOMAIN_NAME=Default
export OS_PROJECT_NAME=admin
export OS_USERNAME=admin
export OS_PASSWORD=ADMIN_PASS
export OS_AUTH_URL=http://controller:5000/v3
export OS_IDENTITY_API_VERSION=3
export OS_IMAGE_API_VERSION=2
```

27. Source the file and test:

```
source ~/admin-openrc
openstack token issue           # should return a valid token
```

NOTE

Always run 'source ~/admin-openrc' at the start of any OpenStack CLI session. Without it, all openstack commands will fail with an authentication error.

11.6 Create Service Project, Demo User, and Role

```
openstack project create --domain default --description 'Service Project' service
openstack project create --domain default --description 'Demo Project' myproject
openstack user create --domain default --password MYUSER_PASS myuser
openstack role create myrole
openstack role add --project myproject --user myuser myrole
```

11.7 Verify Keystone

```
openstack service list
openstack endpoint list
openstack project list
openstack user list
```

Lab Report Q11

Explain the concepts of project, user, role, and endpoint in Keystone. What is a service catalogue and why is it central to how OpenStack services communicate? What is the Fernet token provider and why is it preferred over UUID tokens?

Lab 12: Glance — Image Service and Placement Service

Duration: 2 hours

Objectives

- Install and configure Glance on the Controller.
- Upload a cloud image to Glance and verify it.
- Install and configure the Placement service.
- Verify Placement is operational.

12.1 Create the Glance Database and Credentials

```
source ~/admin-openrc
mysql
CREATE DATABASE glance;
GRANT ALL PRIVILEGES ON glance.* TO 'glance'@'localhost' IDENTIFIED BY
'GLANCE_DBPASS';
GRANT ALL PRIVILEGES ON glance.* TO 'glance'@'%' IDENTIFIED BY 'GLANCE_DBPASS';
FLUSH PRIVILEGES; EXIT;
openstack user create --domain default --password GLANCE_PASS glance
openstack role add --project service --user glance admin
openstack service create --name glance --description 'OpenStack Image' image
openstack endpoint create --region RegionOne image public http://controller:9292
openstack endpoint create --region RegionOne image internal http://controller:9292
openstack endpoint create --region RegionOne image admin http://controller:9292
```

12.2 Install and Configure Glance

```
sudo apt install -y glance
```

28. Edit `/etc/glance/glance-api.conf`. Add or update the following sections:

```
[database]
connection = mysql+pymysql://glance:GLANCE_DBPASS@controller/glance
[keystone_auth]
www_authenticate_uri = http://controller:5000
auth_url = http://controller:5000
memcached_servers = controller:11211
auth_type = password
project_domain_name = Default
user_domain_name = Default
project_name = service
username = glance
password = GLANCE_PASS
[paste_deploy]
flavor = keystone
[glance_store]
stores = file,http
default_store = file
filesystem_store_datadir = /var/lib/glance/images/
```

29. Set correct ownership on the images directory:

```
sudo chown -R glance:glance /var/lib/glance/images/
sudo su -s /bin/sh -c "glance-manage db_sync" glance
sudo service glance-api restart
sudo service glance-api status      # confirm active (running)
```

12.3 Upload a Cloud Image

30. Download the CirrOS test image (very small — ideal for lab testing):

```
wget https://download.cirros-cloud.net/0.6.2/cirros-0.6.2-x86_64-disk.img
```

31. Upload to Glance:

```
openstack image create cirros \
  --file cirros-0.6.2-x86_64-disk.img \
  --disk-format qcow2 \
  --container-format bare \
  --public
openstack image list           # cirros should show as active
```

NOTE

CirrOS is a minimal Linux image designed for cloud testing — it boots in seconds and uses only 12 MB. It is much more suitable for lab instances than a full Ubuntu cloud image. Default login: cirros / gocubsgo

12.4 Install and Configure Placement

```
mysql
CREATE DATABASE placement;
GRANT ALL PRIVILEGES ON placement.* TO 'placement'@'localhost' IDENTIFIED BY
'PLACEMENT_DBPASS';
GRANT ALL PRIVILEGES ON placement.* TO 'placement'@'%' IDENTIFIED BY
'PLACEMENT_DBPASS';
FLUSH PRIVILEGES; EXIT;
openstack user create --domain default --password PLACEMENT_PASS placement
openstack role add --project service --user placement admin
openstack service create --name placement --description 'Placement API' placement
openstack endpoint create --region RegionOne placement public http://controller:8778
openstack endpoint create --region RegionOne placement internal http://controller:8778
openstack endpoint create --region RegionOne placement admin http://controller:8778
sudo apt install -y placement-api
```

32. Edit `/etc/placement/placement.conf`:

```
[api]
auth_strategy = keystone
[keystone_authtoken]
auth_url = http://controller:5000/v3
memcached_servers = controller:11211
auth_type = password
project_domain_name = Default
user_domain_name = Default
project_name = service
username = placement
password = PLACEMENT_PASS
[placement_database]
connection = mysql+pymysql://placement:PLACEMENT_DBPASS@controller/placement
sudo su -s /bin/sh -c "placement-manage db sync" placement
sudo service apache2 restart
placement-status upgrade check  # all checks should pass
```

**Lab Report
Q12**

What is the role of Glance in the OpenStack compute workflow? Why does Nova need the Placement service before scheduling an instance? What is the difference between the disk format (qcow2) and container format (bare) in a Glance image?

Lab 13: Nova and Neutron — Compute and Networking

Duration: 3 hours

Objectives

- Install and configure Nova on both the Controller and Compute nodes.
- Enable the `br_netfilter` kernel module required by the Neutron Linux Bridge agent.
- Install and configure Neutron with Linux Bridge provider networking on both nodes.
- Install Horizon and verify the full stack via the dashboard.

13.1 Nova — Controller Node

```
mysql
CREATE DATABASE nova_api;
CREATE DATABASE nova;
CREATE DATABASE nova_cell0;
GRANT ALL PRIVILEGES ON nova_api.* TO 'nova'@'localhost' IDENTIFIED BY 'NOVA_DBPASS';
GRANT ALL PRIVILEGES ON nova_api.* TO 'nova'@'%' IDENTIFIED BY 'NOVA_DBPASS';
GRANT ALL PRIVILEGES ON nova.* TO 'nova'@'localhost' IDENTIFIED BY 'NOVA_DBPASS';
GRANT ALL PRIVILEGES ON nova.* TO 'nova'@'%' IDENTIFIED BY 'NOVA_DBPASS';
GRANT ALL PRIVILEGES ON nova_cell0.* TO 'nova'@'localhost' IDENTIFIED BY
'NOVA_DBPASS';
GRANT ALL PRIVILEGES ON nova_cell0.* TO 'nova'@'%' IDENTIFIED BY 'NOVA_DBPASS';
FLUSH PRIVILEGES; EXIT;
openstack user create --domain default --password NOVA_PASS nova
openstack role add --project service --user nova admin
openstack service create --name nova --description 'OpenStack Compute' compute
openstack endpoint create --region RegionOne compute public
http://controller:8774/v2.1
openstack endpoint create --region RegionOne compute internal
http://controller:8774/v2.1
openstack endpoint create --region RegionOne compute admin http://controller:8774/v2.1
sudo apt install -y nova-api nova-conductor nova-novncproxy nova-scheduler
```

33. Edit `/etc/nova/nova.conf`. Add or update the following sections. Only sections listed here need to be set — do not remove other existing settings:

```
[DEFAULT]
transport_url = rabbit://openstack:RABBIT_PASS@controller:5672/
my_ip = 10.0.0.11
use_neutron = true
firewall_driver = nova.virt.firewall.NoopFirewallDriver
[api]
auth_strategy = keystone
[keystone_authtoken]
www_authenticate_uri = http://controller:5000/
auth_url = http://controller:5000/
memcached_servers = controller:11211
auth_type = password
project_domain_name = Default
user_domain_name = Default
project_name = service
username = nova
password = NOVA_PASS
[service_user]
send_service_user_token = true
```

```
auth_url = http://controller:5000/identity
auth_strategy = keystone
auth_type = password
project_domain_name = Default
project_name = service
user_domain_name = Default
username = nova
password = NOVA_PASS
[vnc]
enabled = true
server_listen = $my_ip
server_proxyclient_address = $my_ip
[glance]
api_servers = http://controller:9292
[oslo_concurrency]
lock_path = /var/lib/nova/tmp
[placement]
region_name = RegionOne
project_domain_name = Default
project_name = service
auth_type = password
user_domain_name = Default
auth_url = http://controller:5000/v3
username = placement
password = PLACEMENT_PASS
```

34. Populate the databases:

```
sudo su -s /bin/sh -c "nova-manage api_db sync" nova
sudo su -s /bin/sh -c "nova-manage cell_v2 map_cell0" nova
sudo su -s /bin/sh -c "nova-manage cell_v2 create_cell --name=cell1 --verbose" nova
sudo su -s /bin/sh -c "nova-manage db sync" nova
```

35. Restart Nova services:

```
sudo service nova-api restart
sudo service nova-scheduler restart
sudo service nova-conductor restart
sudo service nova-novncproxy restart
```

13.2 Nova — Compute Node

```
# On compute1:
sudo apt install -y nova-compute
```

36. Edit /etc/nova/nova.conf on the compute node with the COMPLETE configuration below — all sections shown are required:

```
[DEFAULT]
transport_url = rabbit://openstack:RABBIT_PASS@controller:5672/
my_ip = 10.0.0.31
use_neutron = true
firewall_driver = nova.virt.firewall.NoopFirewallDriver
[api]
auth_strategy = keystone
[keystone_authtoken]
www_authenticate_uri = http://controller:5000/
auth_url = http://controller:5000/
memcached_servers = controller:11211
```

```
auth_type = password
project_domain_name = Default
user_domain_name = Default
project_name = service
username = nova
password = NOVA_PASS
[service_user]
send_service_user_token = true
auth_url = http://controller:5000/identity
auth_strategy = keystone
auth_type = password
project_domain_name = Default
project_name = service
user_domain_name = Default
username = nova
password = NOVA_PASS
[vnc]
enabled = true
server_listen = 0.0.0.0
server_proxyclient_address = $my_ip
novncproxy_base_url = http://controller:6080/vnc_auto.html
[glance]
api_servers = http://controller:9292
[oslo_concurrency]
lock_path = /var/lib/nova/tmp
[placement]
region_name = RegionOne
project_domain_name = Default
project_name = service
auth_type = password
user_domain_name = Default
auth_url = http://controller:5000/v3
username = placement
password = PLACEMENT_PASS
```

37. Check if hardware virtualisation is available on the compute node:

```
egrep -c '(vmx|svm)' /proc/cpuinfo
```

If the result is 0, the CPU does not support hardware virtualisation (common in nested VM setups).
Edit `/etc/nova/nova-compute.conf`:

```
[libvirt]
virt_type = qemu
sudo service nova-compute restart
```

38. Back on the controller, discover the compute host:

```
# On controller:
sudo su -s /bin/sh -c "nova-manage cell_v2 discover_hosts --verbose" nova
openstack compute service list # nova-compute on compute1 should appear
```

13.3 Enable Kernel Networking Modules — Both Nodes

The Neutron Linux Bridge agent requires the `br_netfilter` kernel module and specific `sysctl` settings. Perform the following on BOTH the controller and compute nodes.

39. Load the module and make it persistent:

```
sudo modprobe br_netfilter
```

```
echo 'br_netfilter' | sudo tee /etc/modules-load.d/br_netfilter.conf
```

40. Set the required sysctl values:

```
sudo nano /etc/sysctl.d/99-neutron-bridge.conf
```

Add:

```
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.conf.all.rp_filter = 0
sudo sysctl --system          # apply settings
sysctl net.bridge.bridge-nf-call-iptables # should return 1
```

13.4 Neutron — Controller Node

```
mysql
CREATE DATABASE neutron;
GRANT ALL PRIVILEGES ON neutron.* TO 'neutron'@'localhost' IDENTIFIED BY
'NEUTRON_DBPASS';
GRANT ALL PRIVILEGES ON neutron.* TO 'neutron'@'%' IDENTIFIED BY 'NEUTRON_DBPASS';
FLUSH PRIVILEGES; EXIT;
openstack user create --domain default --password NEUTRON_PASS neutron
openstack role add --project service --user neutron admin
openstack service create --name neutron --description 'OpenStack Networking' network
openstack endpoint create --region RegionOne network public http://controller:9696
openstack endpoint create --region RegionOne network internal http://controller:9696
openstack endpoint create --region RegionOne network admin http://controller:9696
sudo apt install -y neutron-server neutron-plugin-ml2 neutron-linuxbridge-agent
neutron-dhcp-agent neutron-metadata-agent
```

41. Edit /etc/neutron/neutron.conf:

```
[DEFAULT]
core_plugin = ml2
service_plugins =
transport_url = rabbit://openstack:RABBIT_PASS@controller
auth_strategy = keystone
notify_nova_on_port_status_changes = true
notify_nova_on_port_data_changes = true
[keystone_authtoken]
www_authenticate_uri = http://controller:5000
auth_url = http://controller:5000
memcached_servers = controller:11211
auth_type = password
project_domain_name = Default
user_domain_name = Default
project_name = service
username = neutron
password = NEUTRON_PASS
[nova]
auth_url = http://controller:5000
auth_type = password
project_domain_name = Default
user_domain_name = Default
region_name = RegionOne
project_name = service
username = nova
password = NOVA_PASS
[oslo_concurrency]
```

```
lock_path = /var/lib/neutron/tmp
```

42. Edit /etc/neutron/plugins/ml2/ml2_conf.ini:

```
[ml2]
type_drivers = flat,vlan
tenant_network_types =
mechanism_drivers = linuxbridge
extension_drivers = port_security
[ml2_type_flat]
flat_networks = provider
[securitygroup]
enable_ipset = true
```

43. Edit /etc/neutron/plugins/ml2/linuxbridge_agent.ini. Replace PROVIDER_INTERFACE_NAME with your actual provider interface name (e.g. enp0s8):

```
[linux_bridge]
physical_interface_mappings = provider:enp0s8
[vxlan]
enable_vxlan = false
[securitygroup]
enable_security_group = true
firewall_driver = neutron.agent.linux.iptables_firewall.IptablesFirewallDriver
```

44. Edit /etc/neutron/dhcp_agent.ini:

```
[DEFAULT]
interface_driver = linuxbridge
dhcp_driver = neutron.agent.linux.dhcp.Dnsmasq
enable_isolated_metadata = true
```

45. Edit /etc/neutron/metadata_agent.ini:

```
[DEFAULT]
nova_metadata_host = controller
metadata_proxy_shared_secret = METADATA_SECRET
```

46. Update /etc/nova/nova.conf on the controller to enable Neutron integration. Add this section:

```
[neutron]
auth_url = http://controller:5000
auth_type = password
project_domain_name = Default
user_domain_name = Default
region_name = RegionOne
project_name = service
username = neutron
password = NEUTRON_PASS
service_metadata_proxy = true
metadata_proxy_shared_secret = METADATA_SECRET
```

47. Populate the Neutron database and restart services:

```
sudo su -s /bin/sh -c "neutron-db-manage --config-file /etc/neutron/neutron.conf --
config-file /etc/neutron/plugins/ml2/ml2_conf.ini upgrade head" neutron
sudo service nova-api restart
sudo service neutron-server restart
sudo service neutron-linuxbridge-agent restart
sudo service neutron-dhcp-agent restart
sudo service neutron-metadata-agent restart
```

13.5 Neutron — Compute Node

```
# On compute1:
sudo apt install -y neutron-linuxbridge-agent
```

48. Edit `/etc/neutron/neutron.conf` on the compute node with the COMPLETE configuration below:

```
[DEFAULT]
transport_url = rabbit://openstack:RABBIT_PASS@controller
auth_strategy = keystone
[keystone_authtoken]
www_authenticate_uri = http://controller:5000
auth_url = http://controller:5000
memcached_servers = controller:11211
auth_type = password
project_domain_name = Default
user_domain_name = Default
project_name = service
username = neutron
password = NEUTRON_PASS
[oslo_concurrency]
lock_path = /var/lib/neutron/tmp
```

49. Edit `/etc/neutron/plugins/ml2/linuxbridge_agent.ini` on the compute node. Use the same content as the controller but confirm the provider interface name is correct for this machine:

```
[linux_bridge]
physical_interface_mappings = provider:enp0s8
[vxlan]
enable_vxlan = false
[securitygroup]
enable_security_group = true
firewall_driver = neutron.agent.linux.iptables_firewall.IptablesFirewallDriver
```

50. Add the `[neutron]` section to `/etc/nova/nova.conf` on the compute node (same as controller but without `metadata_proxy` settings):

```
[neutron]
auth_url = http://controller:5000
auth_type = password
project_domain_name = Default
user_domain_name = Default
region_name = RegionOne
project_name = service
username = neutron
password = NEUTRON_PASS
sudo service nova-compute restart
sudo service neutron-linuxbridge-agent restart
```

13.6 Verify Nova and Neutron

```
# On controller:
source ~/admin-openrc
openstack compute service list
openstack network agent list
```

All services should show State: up and Status: enabled. nova-compute should appear on host compute1. The linuxbridge-agent should appear on both controller and compute1.

13.7 Install Horizon Dashboard — Controller Only

```
sudo apt install -y openstack-dashboard
```

51. Edit `/etc/openstack-dashboard/local_settings.py`. Find and update the following settings:

```
OPENSTACK_HOST = "controller"
OPENSTACK_KEYSTONE_URL = "http://controller:5000/v3"
ALLOWED_HOSTS = ['*']
SESSION_ENGINE = 'django.contrib.sessions.backends.cache'
CACHES = {
    'default': {
        'BACKEND': 'django.core.cache.backends.memcached.PyMemcacheCache',
        'LOCATION': 'controller:11211',
    }
}
OPENSTACK_KEYSTONE_MULTIDOMAIN_SUPPORT = True
OPENSTACK_API_VERSIONS = {
    'identity': 3,
    'image': 2,
    'volume': 3,
}
OPENSTACK_KEYSTONE_DEFAULT_DOMAIN = "Default"
OPENSTACK_KEYSTONE_DEFAULT_ROLE = "user"
TIME_ZONE = "Africa/Blantyre"
```

52. Reload Apache (not restart — reload is sufficient and faster):

```
sudo service apache2 reload
```

53. Open a browser from the host machine and navigate to `http://10.0.0.11/horizon`. Log in with:

- Domain: Default
- Username: admin
- Password: ADMIN_PASS (the value you set during keystone-manage bootstrap)

Lab Report Q13

Trace the sequence of API calls when a user clicks 'Launch Instance' in Horizon. Name every OpenStack service involved (Keystone, Placement, Nova, Neutron, Glance) and describe what each one does at that moment. Why must the `br_netfilter` module be loaded before the Neutron Linux Bridge agent starts?

Lab 14: Capstone — Launch Instances and Verify the Cloud

Duration: 3 hours

Objectives

- Create a provider network and subnet in Neutron.
- Create a flavor, key pair, and security group rules.
- Launch two instances (web-server and db-server) using the CirrOS image.
- Verify console access, SSH access, and inter-instance connectivity.
- Explore the deployment through Horizon.

14.1 Create the Provider Network

```
source ~/admin-openrc
openstack network create --share --external \
  --provider-physical-network provider \
  --provider-network-type flat \
  provider
```

54. Create the subnet. Adjust the range, gateway, and DNS to match your physical lab LAN:

```
openstack subnet create \
  --network provider \
  --allocation-pool start=10.0.0.200,end=10.0.0.250 \
  --dns-nameserver 8.8.8.8 \
  --gateway 10.0.0.1 \
  --subnet-range 10.0.0.0/24 \
  provider-subnet
```

NOTE

The allocation pool (10.0.0.200–250) must not overlap with the static IPs already assigned to the controller (10.0.0.11) or compute node (10.0.0.31). Adjust to match your actual lab network.

14.2 Create a Flavor

```
openstack flavor create --id 0 --vcpus 1 --ram 512 --disk 1 m1.nano
```

NOTE

512 MB RAM is used here instead of 64 MB — CirrOS is lightweight but Nova requires at least 256 MB for stable scheduling. Adjust based on your available memory.

14.3 Create Key Pair and Security Group Rules

55. Generate an SSH key pair if one does not already exist:

```
ssh-keygen -t rsa -b 2048 -f ~/.ssh/id_rsa -N ""
openstack keypair create --public-key ~/.ssh/id_rsa.pub lab-key
openstack keypair list          # confirm lab-key appears
```

56. Add rules to the default security group to allow ICMP (ping) and SSH:

```
openstack security group rule create --proto icmp default
openstack security group rule create --proto tcp --dst-port 22 default
```

57. Also allow HTTP for the web-server instance:

```
openstack security group rule create --proto tcp --dst-port 80 default
```

14.4 Launch Instance 1 — Web Server

```
openstack server create \  
  --flavor m1.nano \  
  --image cirros \  
  --nic net-id=$(openstack network show provider -f value -c id) \  
  --security-group default \  
  --key-name lab-key \  
  web-server  
openstack server list           # wait until status shows ACTIVE
```

14.5 Launch Instance 2 — Database Server

```
openstack server create \  
  --flavor m1.nano \  
  --image cirros \  
  --nic net-id=$(openstack network show provider -f value -c id) \  
  --security-group default \  
  --key-name lab-key \  
  db-server  
openstack server list           # both instances should show ACTIVE
```

14.6 Verify Console Access

```
openstack console url show web-server  # get VNC URL
```

58. Open the VNC URL in a browser. You will see the CirrOS login prompt. Log in with:

- Username: cirros
- Password: gocubsgo

59. Inside the web-server instance:

```
ip addr show           # note the IP assigned by Neutron DHCP (10.0.0.2xx)  
ping -c 4 8.8.8.8      # internet access through the provider network
```

14.7 Verify SSH Access from the Controller

60. Get the IP addresses of both instances:

```
openstack server list --long  # shows IP addresses
```

61. SSH from the controller node using the key pair (CirrOS uses the cirros user):

```
ssh -i ~/.ssh/id_rsa cirros@<web-server-ip>
```

62. From inside web-server, ping db-server to verify inter-instance connectivity:

```
ping -c 4 <db-server-ip>
```

14.8 Explore via Horizon

63. In the Horizon dashboard on <http://10.0.0.11/horizon>:

64. Navigate to Compute > Instances. Confirm both instances show as Running.

65. Click the web-server instance. View the Overview, Log, and Console tabs.

66. Navigate to Network > Network Topology. Take a screenshot showing both instances connected to the provider network.

Capstone Report Requirements

1. Screenshot: 'openstack compute service list' — all services State: up.
2. Screenshot: 'openstack network agent list' — all agents State: up.
3. Screenshot: Horizon Compute > Instances — both instances Active.
4. Screenshot: VNC console of web-server showing successful CirrOS login.
5. Screenshot: 'ping <db-server-ip>' from inside web-server — showing replies.
6. Screenshot: Horizon Network > Network Topology.
7. Architecture diagram: two-node layout with IPs, hostnames, and services per node.
8. Reflection (minimum 400 words): challenges encountered, what you would change in a production deployment, and a comparison of this setup to launching an instance on AWS or Azure.

**Lab Report
Q14**

Explain what happens inside OpenStack from the moment 'openstack server create' is executed until the instance reaches ACTIVE status. Name the API calls to Keystone, Nova, Placement, Neutron, and Glance in order, and describe what each one does.

Appendix A: Password Reference

Replace all placeholder passwords with strong passwords before beginning installation. Store them in a secure location — each appears in multiple configuration files across both nodes.

Placeholder	Used for	Files it appears in
ADMIN_PASS	Keystone admin user	~/admin-openrc, keystone-manage bootstrap
RABBIT_PASS	RabbitMQ openstack user	nova.conf (both nodes), neutron.conf (both nodes)
KEYSTONE_DBPASS	Keystone MariaDB user	keystone.conf
GLANCE_DBPASS	Glance MariaDB user	glance-api.conf
GLANCE_PASS	Glance service user in Keystone	glance-api.conf
NOVA_DBPASS	Nova MariaDB user	nova.conf (controller only)
NOVA_PASS	Nova service user in Keystone	nova.conf (both nodes), neutron.conf (both nodes)
NEUTRON_DBPASS	Neutron MariaDB user	neutron.conf (controller only)
NEUTRON_PASS	Neutron service user in Keystone	neutron.conf (both nodes), nova.conf (both nodes)
PLACEMENT_DBPASS	Placement MariaDB user	placement.conf
PLACEMENT_PASS	Placement service user in Keystone	placement.conf, nova.conf (both nodes)
METADATA_SECRET	Neutron metadata proxy shared secret	metadata_agent.ini, nova.conf (both nodes)
MYUSER_PASS	Demo user myuser password	set during openstack user create only

Appendix B: Service Port Reference

Service	Port	Protocol	Node
Keystone (Identity)	5000	HTTP	Controller
Glance (Image)	9292	HTTP	Controller
Nova API (Compute)	8774	HTTP	Controller
Nova VNC Proxy	6080	HTTP/WebSocket	Controller
Neutron (Networking)	9696	HTTP	Controller
Placement API	8778	HTTP	Controller

Horizon (Dashboard)	80	HTTP	Controller
MariaDB	3306	TCP	Controller
RabbitMQ	5672	AMQP	Controller
Memcached	11211	TCP	Controller
Etcd (client)	2379	HTTP	Controller
Etcd (peer)	2380	HTTP	Controller

Appendix C: Common Troubleshooting

Problem	Likely Cause	Fix
Cannot ping VM from host (NAT adapter)	NAT is designed for outbound only — this is expected	Use the Host-Only adapter (enp0s8) IP instead — always 192.168.56.x
Cloned VM gets the same IP as original	machine-id and/or 50-cloud-init.yaml were cloned unchanged — DHCP sees the same client	On the clone: <code>sudo truncate -s 0 /etc/machine-id && sudo rm -f /var/lib/dbus/machine-id</code> — then reboot. Do NOT run <code>systemd-machine-id-setup</code> .
enp0s8 has no IP after boot	Host-Only DHCP not enabled in VirtualBox Host Network Manager	File > Host Network Manager > vboxnet0 > DHCP Server tab > Enable
docker: permission denied	User not yet in the docker group in this session	Run: <code>newgrp docker</code> OR log out and log back in
minikube start: permission denied	User not in docker group	<code>sudo usermod -aG docker \$USER</code> then log out/in
K8s pod shows ImagePullBackOff	Image name wrong or image not pushed to public registry	Check <code>docker pull <image></code> works. Ensure Docker Hub repo is Public.
Rolling update does not change page content	v2 image was not built or not pushed to Docker Hub	Build and push v2 as shown in Lab 6.4 before running Lab 8.5
ping compute1 fails from controller	Hostname not in /etc/hosts or wrong IP	Check /etc/hosts on both nodes. Confirm IP with: <code>ip addr show enp0s3</code>
mysql: Access denied for root	Ubuntu 22.04 MariaDB uses unix_socket auth	Use: <code>sudo mysql</code> (not <code>mysql -u root -p</code>). No password needed for root locally.
neutron-linuxbridge-agent fails to start	br_netfilter module not loaded	Run: <code>sudo modprobe br_netfilter</code> then restart the agent
Nova compute not discovered	nova-compute service not running on compute1, or clock skew	Check: <code>sudo service nova-compute status</code> . Also check NTP: <code>chronyc tracking</code>
Horizon 500 error on login	Memcached not running or CACHES setting wrong in local_settings.py	Check: <code>sudo service memcached status</code> . Confirm <code>-l 10.0.0.11</code> in /etc/memcached.conf

Instance stuck in BUILD

Nova cannot schedule —
compute host not discovered
or Placement not configured

Run: nova-manage cell_v2 discover_hosts -
-verbose. Check placement-status upgrade
check

— *End of Laboratory Instruction Guide* —